

Universidade Estadual de Londrina

Sistemas Distribuídos

Análise Global de Mudanças Climáticas e Eventos Extremos

Processamento de Big Data com Apache Spark

Marco Antônio Cottorello Henry e Danilo Kotaka Marana

Londrina

26 de maio de 2026

Sumário

1	Introdução	2
2	Arquitetura do Cluster	3
2.1	Trecho do <code>docker-compose.yml</code>	3
3	Metodologia de ETL	4
3.1	Leitura com inferência de esquema	4
3.2	Remoção de nulos, conversão de datas e limpeza de coordenadas	5
3.3	Aplicação de <code>cache</code>	5
3.4	Leitura e limpeza da base de CO ₂	5
3.5	Filtro de incerteza via <i>Window Function</i>	6
4	Análise de Otimização — Cache	7
4.1	O que é <code>.cache()</code> e como funciona	7
4.2	Por que foi aplicado em <code>df_limpo</code>	7
4.3	Comparativo de desempenho	7
4.4	Alternativa: <code>.persist()</code>	8
5	Respostas às Perguntas	8
5.1	P1 — Evolução da temperatura média por década	8
5.2	P2 — 10 anos mais quentes por continente	9
5.3	P3 — Cidades em risco (maior instabilidade)	11
5.4	P4 — Correlação entre máximas e mínimas nos trópicos	12
5.5	P5 — Qualidade dos dados	13
5.6	P6 — Correlação entre emissões de CO ₂ e aquecimento	14
5.6.1	Etapa 1 — Cruzamento das bases (<i>join</i>)	14
5.6.2	Etapa 2 — Correlação das taxas de aumento (resposta à pergunta)	15
5.7	P7 — Aceleração térmica (<i>Window Functions</i>)	16
5.8	P8 — Previsão de temperatura com MLlib	17
6	Evidências do Spark UI	19
6.1	DAG das operações mais pesadas	19
6.2	Estágios, tempo de execução e tarefas	19
7	Dicionário de Dados Final	22

1 Introdução

As mudanças climáticas figuram entre os temas mais relevantes da ciência contemporânea, e o estudo de suas tendências depende da análise de séries históricas extensas de temperatura. Conjuntos de dados como o *Climate Change: Earth Surface Temperature Data*, da Berkeley Earth [Ber17], reúnem medições de mais de dois séculos por cidade e país, alcançando milhões de registros — um volume que torna inviável o processamento em uma única máquina por ferramentas tradicionais. Esse cenário motiva o uso de plataformas de *Big Data* capazes de distribuir o cálculo por um aglomerado de nós.

Este trabalho utiliza o **Apache Spark** [Apa24], motor de processamento distribuído em memória, para investigar tendências de aquecimento e anomalias térmicas ao longo dos últimos 250 anos. O Spark foi implantado em um *cluster* no modelo *master/worker*, totalmente containerizado com Docker e **docker-compose**, o que garante um ambiente reprodutível e isolado. À base de temperaturas somou-se a base de emissões de CO₂ da *Our World in Data* [Our24], permitindo correlacionar aquecimento e emissões por meio de um *join* entre as duas fontes.

O objetivo é demonstrar, na prática, a capacidade do Spark de lidar com transformações em larga escala e cálculos estatísticos complexos, respondendo a oito perguntas analíticas que envolvem agregações por década, rankings de instabilidade climática, correlações, *Window Functions* e previsão com regressão linear via **Spark MLlib**. Ao longo do *pipeline*, explora-se também o processo de **ETL** (limpeza, padronização e enriquecimento dos dados) e técnicas de otimização como o uso de **cache** para reaproveitamento de resultados intermediários.

O restante do relatório está organizado da seguinte forma: a Seção 2 descreve a arquitetura do *cluster*; a Seção 3 detalha a metodologia de ETL; a Seção 4 discute a análise de otimização com **cache**; a Seção 5 apresenta as respostas às oito perguntas com suas visualizações e interpretações; e a Seção 6 reúne as evidências de execução coletadas na Spark UI.

2 Arquitetura do Cluster

A análise foi executada sobre um cluster **Apache Spark 3.5.0** totalmente containerizado com **Docker** e orquestrado via **docker-compose**. Essa escolha garante um ambiente reprodutível e isolado, no qual cada nó do cluster corresponde a um contêiner independente, comunicando-se por uma rede dedicada. O ambiente utiliza **Python 3.8**, versão padrão da imagem base `apache/spark:3.5.0`.

O cluster segue o modelo **master/worker** (mestre/trabalhador) do Spark Standalone e é composto pelos seguintes serviços:

- **1 Spark Master** (`spark-master`): processo coordenador responsável por gerenciar os recursos do cluster e escalonar as aplicações. Expõe a porta **7077** para submissão de jobs e a **8080** para a interface web de monitoramento.
- **3 Spark Workers** (`spark-worker-1`, `-2` e `-3`): nós trabalhadores que executam efetivamente as tarefas. Cada worker é configurado com `SPARK_WORKER_CORES=2` e `SPARK_WORKER_MEMORY=4g`, totalizando **6 cores** e **12 GB** de memória disponíveis para processamento paralelo.
- **1 History Server** (`spark-history`): serviço que lê o diretório de eventos (`spark-events`) e disponibiliza, na porta **18080**, o histórico de aplicações já finalizadas, permitindo a análise *post-mortem* das DAGs e estágios.
- **1 JupyterLab** (`jupyter`): ambiente interativo de desenvolvimento construído a partir de uma imagem customizada (`Dockerfile.jupyter`), baseada em `apache/spark:3.5.0` com a adição das bibliotecas `numpy`, `matplotlib`, `pandas` e `pyspark==3.5.0`. Atua como *driver* da aplicação, conectando-se ao master pela porta **8888**.

Todos os serviços estão conectados a uma rede *bridge* do Docker, denominada **spark-network**, que isola o tráfego do cluster e permite que os contêineres se resolvam entre si por *hostname* (por exemplo, `spark://spark-master:7077`). Volumes compartilhados (`./spark-apps`, `./spark-data` e `./spark-events`) garantem que código, dados de entrada/saída e logs de eventos sejam visíveis de forma consistente em todos os nós.

2.1 Trecho do `docker-compose.yml`

O Código 1 ilustra a definição do *master*, de um *worker* representativo e da rede **spark-network**. Os demais workers são análogos, variando apenas o *hostname* e a porta da interface web.

```
1 services:
2   spark-master:
3     build: .
4     container_name: spark-master
5     hostname: spark-master
6     command: /opt/spark/bin/spark-class
7       org.apache.spark.deploy.master.Master
8     environment:
9       - SPARK_MASTER_HOST=spark-master
10      - SPARK_MASTER_PORT=7077
```

```

10     - SPARK_MASTER_WEBUI_PORT=8080
11   ports:
12     - "8080:8080"
13     - "7077:7077"
14   networks:
15     - spark-network
16
17   spark-worker-1:
18     build: .
19     container_name: spark-worker-1
20     hostname: spark-worker-1
21     command: /opt/spark/bin/spark-class
22               org.apache.spark.deploy.worker.Worker spark://spark-master:7077
23     environment:
24       - SPARK_WORKER_CORES=2
25       - SPARK_WORKER_MEMORY=4g
26       - SPARK_WORKER_WEBUI_PORT=8081
27     depends_on:
28       - spark-master
29     networks:
30       - spark-network
31
32   networks:
33     spark-network:
34       driver: bridge

```

Listing 1: Trecho do docker-compose.yml: master, worker e rede bridge.

3 Metodologia de ETL

O processo de **ETL** (*Extract, Transform, Load*) foi responsável por transformar dados climáticos brutos em um conjunto confiável para análise. A base principal foi o `GlobalLandTemperaturesByCity` complementada pela base de emissões `owid-co2-data.csv`. As etapas são detalhadas a seguir.

3.1 Leitura com inferência de esquema

A ingestão utilizou `spark.read.csv` com `header=True` e `inferSchema=True`. A inferência automática faz o Spark amostrar os dados para deduzir os tipos de cada coluna (datas, double, string), dispensando a definição manual de um `StructType` e simplificando o protótipo.

```

1 df = spark.read.csv(
2     "/opt/spark-data/input/GlobalLandTemperaturesByCity.csv",
3     header=True,
4     inferSchema=True,
5 )

```

Listing 2: Leitura da base de temperaturas com inferência de esquema.

3.2 Remoção de nulos, conversão de datas e limpeza de coordenadas

Registros sem temperatura média (`AverageTemperature` nulo) não agregam valor analítico e foram descartados com `dropna`. A coluna textual `dt` foi convertida para o tipo `date` com `to_date`, e dela extraíram-se as colunas `Year` e `Month` via `year()` e `month()`, facilitando os agrupamentos temporais.

As coordenadas vinham no formato "57.05N" / "10.33E", misturando número e ponto cardinal. Para viabilizar o filtro geográfico das zonas tropicais (P4), aplicou-se `regexp_replace` removendo os caracteres `[NSEW]` e convertendo o resultado para `double`. Todas as transformações foram encadeadas de forma declarativa (*lazy*), construindo o `DataFrame` `df_limpo`.

```
1 df_limpo = (  
2     df.withColumn("dt", to_date(col("dt")))  
3     .withColumn("Year", year(col("dt")))  
4     .withColumn("Month", month(col("dt")))  
5     .dropna(subset=["AverageTemperature"])  
6     .withColumn(  
7         "Latitude", regexp_replace(col("Latitude"), "[NSEW]",  
8             "").cast("double")  
9     )  
10    .withColumn(  
11        "Longitude", regexp_replace(col("Longitude"), "[NSEW]",  
12            "").cast("double")  
13    )  
14 )
```

Listing 3: Conversão de datas, remoção de nulos e limpeza de coordenadas.

3.3 Aplicação de cache

Como `df_limpo` é reutilizado por praticamente todas as perguntas, aplicou-se `cache()` logo após a limpeza. Sem isso, cada ação (`count`, `show`, agregações) reexecutaria todo o pipeline de leitura e transformação a partir do disco. O `cache` materializa o resultado em memória após a primeira ação, e as etapas subsequentes reaproveitam esses dados. A justificativa quantitativa é detalhada na Seção 4.

```
1 df_limpo.show(5)  
2 # Otimizacao: cache no DataFrame limpo, evitando reprocessar a cada nova  
3   acao  
4 df_limpo.cache()
```

Listing 4: Materialização em memória do `DataFrame` limpo.

3.4 Leitura e limpeza da base de CO₂

A base `owid-co2-data.csv` contém, além de países, diversos **agregados** ("World", "Asia (GCP)", blocos regionais e grupos de renda como "High-income countries"). Se mantidos, esses agregados seriam contabilizados em dobro no *join* com a base de temperaturas (que

é granularizada por país), distorcendo a correlação da P6. Por isso foram explicitamente filtrados com `~col("country").isin([...])`. Também se restringiu o período a `year >= 1900` e removeram-se linhas sem valor de `co2`.

```
1 df_co2_clean = (  
2     df_co2.filter(col("year") >= 1900)  
3     .filter(  
4         ~col("country").isin(  
5             [  
6                 "World", "Asia", "Asia (GCP)", "Europe", "Africa",  
7                 "North America", "South America", "Oceania",  
8                 "European Union (27)", "European Union (28)",  
9                 "High-income countries", "Low-income countries",  
10                "Lower-middle-income countries", "Upper-middle-income  
11                countries",  
12                "OECD (GCP)", "Non-OECD (GCP)",  
13                # ... demais blocos regionais e grupos de renda  
14            ]  
15        )  
16    )  
17    .select(  
18        col("country").alias("Country_CO2"),  
19        col("year").alias("Year_CO2"),  
20        col("co2"),  
21        col("co2_per_capita"),  
22        col("total_ghg"),  
23    )  
24    .dropna(subset=["co2"])
```

Listing 5: Filtragem de agregados globais e seleção de colunas da base de CO2.

3.5 Filtro de incerteza via *Window Function*

O *dataset* traz a coluna `AverageTemperatureUncertainty`. Para descartar medições pouco confiáveis, definiu-se uma janela particionada por `City` e `Country` e calculou-se a média histórica de cada cidade (`HistAvg`). Uma *Window Function* retorna um valor agregado para **cada linha** sem colapsar o `DataFrame`, o que permite comparar a incerteza de cada registro com a média histórica da sua própria cidade. Mantiveram-se apenas os registros cuja incerteza fosse menor ou igual a 10% dessa média.

```
1 window_city = Window.partitionBy("City", "Country")  
2  
3 df_limpo_com_histavg = df_limpo.withColumn(  
4     "HistAvg", avg("AverageTemperature").over(window_city)  
5 )  
6  
7 df_filtrado = df_limpo_com_histavg.filter(  
8     col("AverageTemperatureUncertainty") <= (0.10 * col("HistAvg"))  
9 )
```

Listing 6: Definição da janela por cidade e filtro de incerteza relativa.

4 Análise de Otimização — Cache

4.1 O que é `.cache()` e como funciona

No Spark, transformações são **preguiçosas** (*lazy*): apenas constroem um plano lógico, a DAG. Esse plano só é executado quando uma **ação** (`count`, `show`, `collect`, escrita) é disparada. Por padrão, o resultado de um `DataFrame` **não é guardado**; cada nova ação reexecuta toda a cadeia desde a leitura do arquivo.

O método `.cache()` altera esse comportamento. Internamente ele marca o `DataFrame` com o nível de armazenamento `MEMORY_AND_DISK`: na primeira ação subsequente, o Spark calcula o resultado e o **persiste em memória** nos executores (transbordando para disco se faltar RAM). A partir daí, as próximas ações leem diretamente dessas partições materializadas, eliminando a releitura de disco e o reprocessamento das transformações.

4.2 Por que foi aplicado em `df_limpo`

`df_limpo` é o tronco comum de todo o pipeline: a partir dele derivam o filtro de incerteza, a análise por década (P1), o ranking de risco (P3), as zonas tropicais (P4) e demais perguntas. Sem cache, cada uma dessas ramificações refaria a leitura do CSV e toda a limpeza. Persistindo `df_limpo`, paga-se o custo da limpeza **uma única vez** e reaproveita-se o resultado em todas as análises.

4.3 Comparativo de desempenho

Para mensurar o ganho de forma controlada, instrumentou-se o tempo de uma mesma ação (`count`) em dois cenários: a **primeira contagem**, que ainda força a releitura do CSV e toda a cadeia de transformações; e uma contagem **após o cache** estar materializado em memória. Os valores a seguir foram **medidos na execução real** do *notebook* no cluster:

```
1 import time
2
3 # Tempo SEM cache: a 1a contagem reexecuta leitura + limpeza a partir do
  disco
4 _t = time.time()
5 df_limpo.count()
6 tempo_sem_cache = time.time() - _t
7
8 # Aplica cache e materializa
9 df_limpo.cache()
10 df_limpo.count()
11
12 # Tempo COM cache: dados já residentes em memória
13 _t = time.time()
14 df_limpo.count()
15 tempo_com_cache = time.time() - _t
16
17 print(f"Sem cache: {tempo_sem_cache:.2f}s | Com cache:
    {tempo_com_cache:.2f}s")
18 print(f"Ganho: {tempo_sem_cache / tempo_com_cache:.1f}x")
```


Listing 7: Benchmark de cache executado no cluster.

Cenário	Tempo medido	Ganho
Sem cache (1ª contagem)	2,65 s	—
Com cache (em memória)	0,25 s	$\sim 10,7\times$

Tabela 1: Impacto do cache sobre uma ação count em df_limpo.

O ganho de aproximadamente **10,7×** confirma que, em cargas iterativas com múltiplas ações sobre o mesmo DataFrame, o **cache** é determinante. O efeito é ainda mais relevante no contexto completo: o *pipeline* inteiro (ETL + 8 perguntas + 6 gráficos) executou em **~1 min 43 s**, e sem o cache de df_limpo cada uma das oito análises pagaria novamente o custo de releitura e limpeza dos 532 MB. Vale notar que o cache só compensa quando o DataFrame é reutilizado; para um uso único, ele apenas adiciona o custo de materialização.

4.4 Alternativa: .persist()

O método `.cache()` é, na prática, um atalho para `.persist(StorageLevel.MEMORY_AND_DISK)`. Já `.persist()` permite escolher explicitamente o nível de armazenamento — por exemplo, `MEMORY_ONLY` (mais rápido, mas recalcula partições que não cabem na RAM), `MEMORY_AND_DISK_SER` (serializado, menor pegada de memória) ou `DISK_ONLY`. Em cenários com restrição de memória, `persist` com nível serializado pode ser preferível ao `cache` padrão.

5 Respostas às Perguntas

5.1 P1 — Evolução da temperatura média por década

O que foi feito: a partir de df_filtrado, criou-se a coluna Decada truncando o ano (`(Year/10).cast("int")*10`), agrupando-se por década e calculando a temperatura média global de cada uma.

```

1 df_decada = (
2     df_filtrado.withColumn("Decada", (col("Year") / 10).cast("int") * 10)
3     .groupBy("Decada")
4     .agg(_round(avg("AverageTemperature"), 2).alias("AvgGlobalTemp"))
5     .orderBy("Decada")
6 )
7 df_decada.show(truncate=False)

```

Listing 8: Cálculo da temperatura média por década.

Resultado (décadas selecionadas):

- 1900: 18,80 °C

- 1950: 18,73 °C
- 1980: 18,67 °C
- 2000: 19,21 °C
- 2010: 19,45 °C

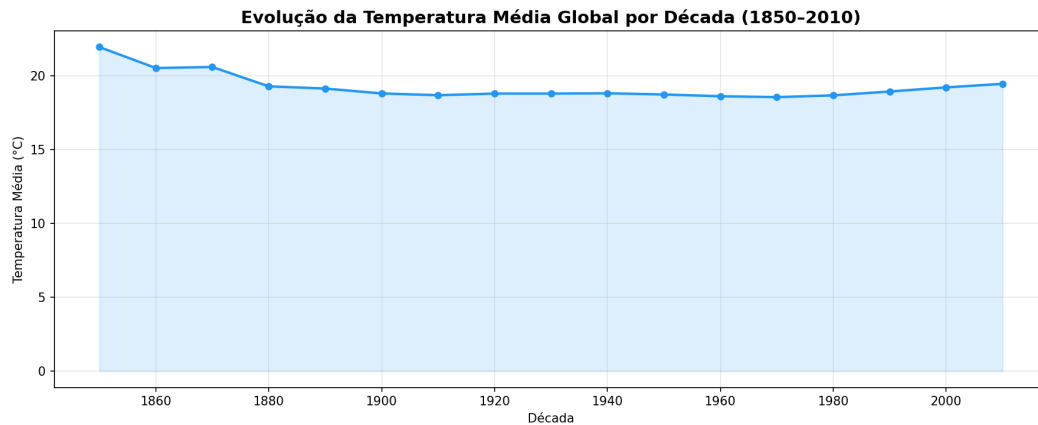


Figura 1: Evolução da temperatura média global por década.

Interpretação: o aquecimento mais pronunciado concentra-se nas décadas finais: entre 1980 (18,67 °C) e 2010 (19,45 °C) há um aumento de $\sim 0,78$ °C, e a curva sobe de forma consistente de 1980 em diante. As décadas intermediárias (1950, 1980) chegam a ficar ligeiramente **abaixo** de 1900, o que não indica resfriamento real, mas sim o **viés de composição** do *dataset*: o conjunto de cidades medidas muda ao longo do tempo, e a média global é sensível a quais estações estão presentes em cada década. A aceleração recente é coerente com o aquecimento global documentado na literatura.

5.2 P2 — 10 anos mais quentes por continente

O que foi feito: restringiram-se os dados aos últimos 50 anos (`Year >= 1974`) e agruparam-se os países em **sete regiões continentais** (América do Sul, América do Norte, América Central e Caribe, Europa, Ásia, África e Oceania). Cada região reúne **praticamente todos os seus países** presentes no *dataset* da ordem de dezenas por região, e não apenas uma amostra. Para cada região, calculou-se a temperatura média anual, ordenando de forma decrescente e tomando o *top 10*.

```

1 continentes = {
2     "America do Sul": ["Brazil", "Argentina", "Chile",
3                       "Colombia",
4                       "Peru", "Venezuela", "Ecuador",
5                       "Bolivia", ...],
6     "America do Norte": ["United States", "Canada", "Mexico"],
7     "America Central e Caribe": ["Guatemala", "Honduras", "Cuba", "Haiti",
8                                 "Dominican Republic", "Jamaica", ...],

```

```

7     "Europa": ["Germany", "France", "United Kingdom",
8               "Sweden", "Norway", "Poland",
9               "Ukraine", ...],
10    "Asia": ["China", "India", "Japan", "Russia",
11            "Turkey", "Saudi Arabia", "Kazakhstan",
12            ...],
13    "Africa": ["Nigeria", "South Africa", "Egypt",
14              "Algeria", "Morocco", "Sudan",
15              "Angola", ...],
16    "Oceania": ["Australia", "New Zealand", "Papua New
17                Guinea"],
18 }
19
20 df_recente = df_filtrado.filter(col("Year") >= 1974)
21 for continente, paises in continentes.items():
22     df_cont = (
23         df_recente.filter(col("Country").isin(paises))
24         .groupBy("Year")
25         .agg(_round(avg("AverageTemperature"), 2).alias("AvgTemp"))
26         .orderBy(col("AvgTemp").desc())
27         .limit(10)
28     )
29     df_cont.show()

```

Listing 9: Ranking dos anos mais quentes por região continental.

Resultado (3 anos mais quentes e pico de temperatura por região):

- **América Central e Caribe:** 1998, 1997, 2003 (pico $\sim 25,9^{\circ}\text{C}$)
- **África:** 2010, 2005, 1998 (pico $\sim 24,4^{\circ}\text{C}$)
- **América do Sul:** 1998, 2002, 2012 (pico $\sim 22,2^{\circ}\text{C}$)
- **Ásia:** 2013, 2010, 2012 (pico $\sim 22,1^{\circ}\text{C}$)
- **América do Norte:** 2013, 2012, 1998 (pico $\sim 17,5^{\circ}\text{C}$)
- **Oceania:** 2013, 1998, 2010 (pico $\sim 16,9^{\circ}\text{C}$)
- **Europa:** 2007, 2011, 2013 (pico $\sim 11,5^{\circ}\text{C}$)

Interpretação: os anos mais quentes concentram-se majoritariamente após 1998, reforçando o aquecimento recente. Anos como **1998** e **2013** aparecem no topo de quase todas as regiões. A ordenação por temperatura absoluta segue a latitude: **América Central e Caribe** e **África** (faixas tropicais) lideram, enquanto **Europa** fica por último — mais fria agora que a lista inclui países nórdicos e do leste europeu. Note que a média da Europa caiu em relação a uma amostra menor de países, ilustrando como a composição do conjunto afeta o valor absoluto.

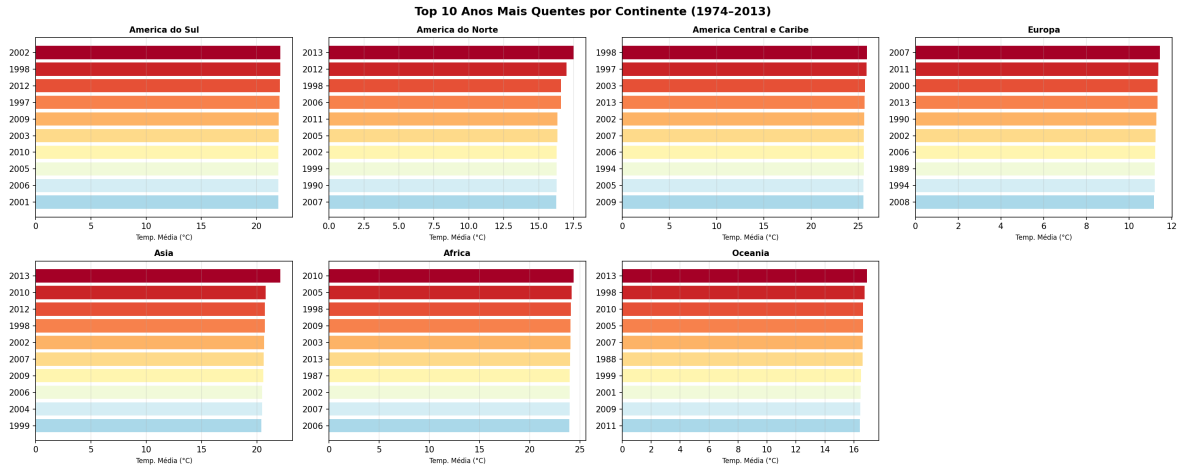


Figura 2: Top 10 anos mais quentes por continente (1974–2013).

Limitação metodológica: o *dataset* não possui uma coluna de continente; o agrupamento por região foi montado manualmente. Embora agora abranja a quase totalidade dos países de cada região (tornando as médias bem mais representativas que um recorte pequeno), permanecem dois vieses: países sem registros no *dataset* não entram, e a média é simples por país/ano — não ponderada por área ou número de estações.

5.3 P3 — Cidades em risco (maior instabilidade)

O que foi feito: considerando o último século (`Year >= 1920`), agruparam-se os dados por cidade/país e calculou-se o **desvio padrão** da temperatura, indicador de volatilidade climática. O *top 10* representa as cidades de clima mais instável.

```
1 df_risk = (
2     df_filtrado.filter(col("Year") >= 1920)
3     .groupBy("City", "Country")
4     .agg(_round(stddev("AverageTemperature"),
5               2).alias("Temperature_StdDev"))
6     .orderBy(col("Temperature_StdDev").desc())
7     .limit(10)
8 )
9 df_risk.show()
```

Listing 10: Cidades com maior desvio padrão de temperatura.

Resultado: a cidade de maior instabilidade é **Yichun (China)**, com desvio padrão de **14,34 °C**. As dez primeiras colocadas são todas cidades do **nordeste da China**.

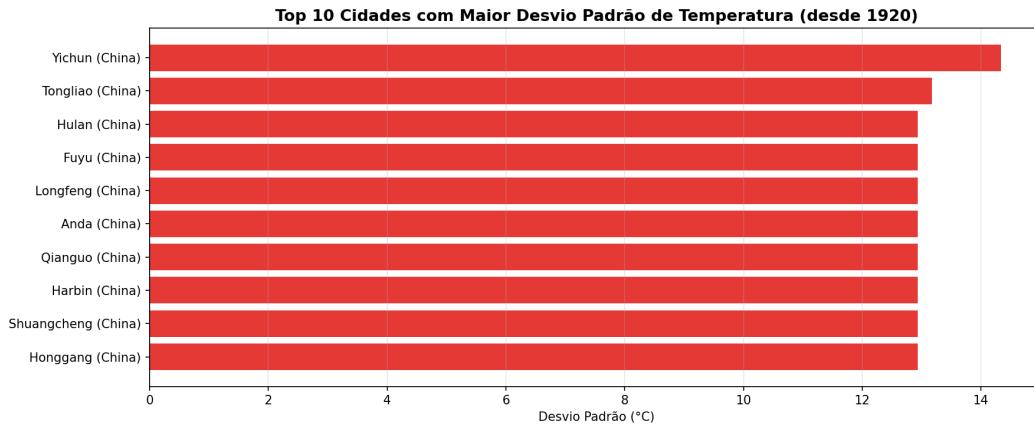


Figura 3: Top 10 cidades com maior desvio padrão de temperatura.

Interpretação: o domínio de cidades do nordeste chinês reflete o **clima continental extremo** da região, caracterizado por invernos rigorosos e verões quentes, o que produz amplitudes térmicas anuais muito elevadas.

5.4 P4 — Correlação entre máximas e mínimas nos trópicos

O que foi feito: filtraram-se as cidades na faixa tropical (Latitude entre $-23,5$ e $23,5$) e, por cidade/ano, calcularam-se a temperatura máxima (T_Max) e mínima (T_Min) mensais. Em seguida, mediu-se o coeficiente de correlação de Pearson entre elas com `stat.corr`.

```

1 df_tropicos = (
2     df_filtrado.filter((col("Latitude") >= -23.5) & (col("Latitude") <=
3         23.5))
4     .groupBy("City", "Year")
5     .agg(
6         _max("AverageTemperature").alias("T_Max"),
7         _min("AverageTemperature").alias("T_Min"),
8     )
9 )
10 corr_t = df_tropicos.stat.corr("T_Max", "T_Min")

```

Listing 11: Correlação entre T_Max e T_Min nas zonas tropicais.

Resultado: coeficiente de Pearson de **0,4026**.

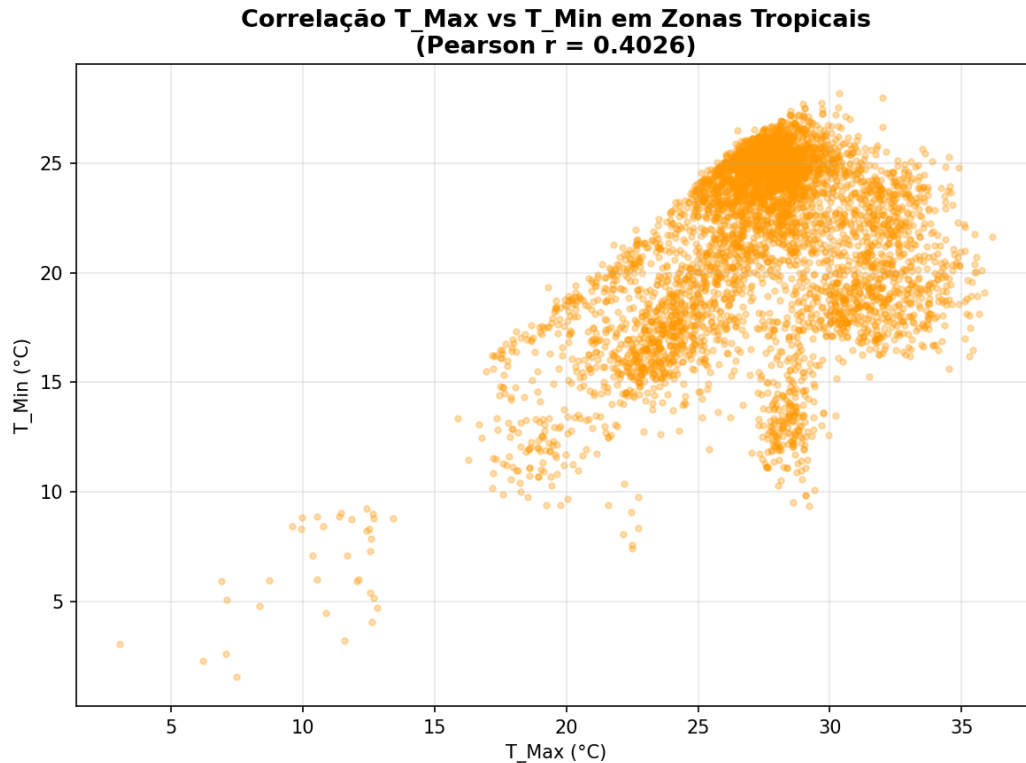


Figura 4: Dispersão entre temperaturas máximas e mínimas nas zonas tropicais.

Interpretação: há uma correlação **moderada positiva** entre máximas e mínimas. Faz sentido nos trópicos: anos mais quentes tendem a elevar tanto as máximas quanto as mínimas, mas a correlação não é forte porque outros fatores (umidade, sazonalidade, altitude) também influenciam.

5.5 P5 — Qualidade dos dados

O que foi feito: usando a média histórica por cidade, removeram-se os registros cuja `AverageTemperatureUncertainty` excedia 10% da média da cidade, e agruparam-se os removidos por país.

```
1 df_alta_incerteza = df_limpo_com_histavg.filter(
2     col("AverageTemperatureUncertainty") > (0.10 * col("HistAvg")))
3 )
4 df_alta_incerteza.groupBy("Country").count().orderBy(col("count").desc()).show(10)
```

Listing 12: Registros de alta incerteza removidos, agrupados por país.

Resultado: os países com mais registros descartados foram **Rússia** (344.520), **China** (256.988) e **EUA** (191.539). Ao todo, cerca de **1.938.936** registros foram removidos — o conjunto passou de **8.235.082** para **6.296.146** linhas.

Interpretação: a concentração de descartes em países de grande extensão territorial e com

séries históricas mais antigas é esperada: medições remotas e instrumentos antigos carregam maior incerteza. O filtro removeu $\sim 23,5\%$ dos dados, melhorando a confiabilidade das análises seguintes ao custo de algum volume.

5.6 P6 — Correlação entre emissões de CO₂ e aquecimento

A pergunta do enunciado é precisa: existe correlação entre o **aumento das emissões** de CO₂ de um país e o **aumento da sua temperatura** nos últimos 50 anos? A palavra-chave é *aumento* — trata-se de correlacionar **tendências** (variação ao longo do tempo), e não níveis absolutos. Por isso a questão foi abordada em duas etapas.

5.6.1 Etapa 1 — Cruzamento das bases (*join*)

Primeiro agregou-se a temperatura média por país/ano (últimos 50 anos) e fez-se um *inner join* com a base de CO₂ limpa pelas chaves país e ano, conforme exigido pelo desafio. Uma correlação ingênua sobre os **níveis absolutos** serve apenas como diagnóstico inicial.

```
1 df_temp_country = (  
2     df_filtrado.filter(col("Year") >= 1974)  
3     .groupBy("Country", "Year")  
4     .agg(avg("AverageTemperature").alias("AvgTemp"))  
5 )  
6  
7 df_join = df_temp_country.join(  
8     df_co2_clean,  
9     (df_temp_country["Country"] == df_co2_clean["Country_CO2"])  
10    & (df_temp_country["Year"] == df_co2_clean["Year_CO2"]),  
11    "inner",  
12 ).dropna(subset=["co2", "AvgTemp"])  
13  
14 corr_nivel = df_join.stat.corr("co2", "AvgTemp")    # -0.1727
```

Listing 13: Join entre temperatura e emissões; correlação de níveis (diagnóstico).

Resultado da Etapa 1: Pearson de $-0,1727$ sobre o *join* de **5.829** registros. Esse valor é **enganoso**: correlacionar emissão absoluta com a temperatura *média* do país sofre de **viés de confundimento geográfico** — grandes emissores como Rússia e Canadá são países frios, então a latitude domina o sinal. Esse número **não responde** à pergunta, que é sobre *aumento* e não sobre nível.

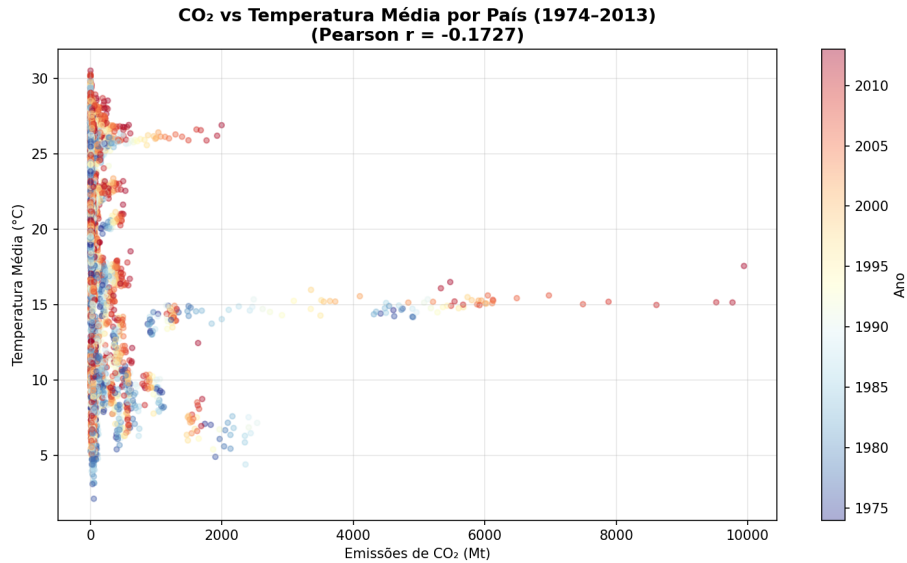


Figura 5: Etapa 1 (diagnóstico): emissão absoluta vs temperatura média por país — correlação distorcida pela geografia.

5.6.2 Etapa 2 — Correlação das taxas de aumento (resposta à pergunta)

Para responder de fato, calculou-se, para **cada país**, a **tendência** (taxa de variação anual) tanto da temperatura quanto das emissões, via inclinação da regressão linear — obtida em forma fechada por $\text{slope} = \text{cov}(\text{ano}, x) / \text{var}(\text{ano})$, usando as funções `covar_pop` e `var_pop` sobre uma janela por país. Em seguida correlacionaram-se as duas tendências **através dos países** (exigindo no mínimo 10 anos de dados por país).

```

1 from pyspark.sql.functions import covar_pop, var_pop, count
2
3 slopes = (
4     df_join.groupBy("Country").agg(
5         (covar_pop("Year", "AvgTemp") /
6          var_pop("Year")).alias("slope_temp"),
7         (covar_pop("Year", "co2") / var_pop("Year")).alias("slope_co2"),
8         count("*").alias("n"),
9     ).filter(col("n") >= 10)
10
11 corr_variacao = slopes.stat.corr("slope_temp", "slope_co2")    # -0.0017

```

Listing 14: Tendência (slope) por país e correlação dos aumentos.

Resultado da Etapa 2: Pearson de $-0,0017$ entre as taxas de aumento, sobre **147 países** — ou seja, **correlação praticamente nula**.

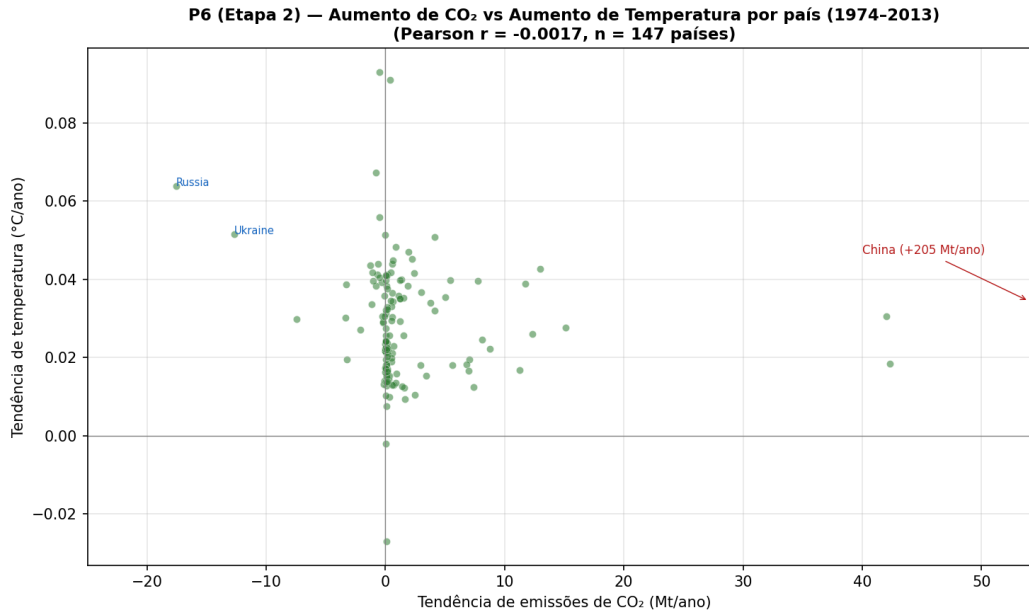


Figura 6: Etapa 2 (resposta): tendência de emissões vs tendência de temperatura por país. Quase todos os países aquecem ($y > 0$) independentemente da trajetória de emissões.

Interpretação: ao nível de país, **não há correlação linear** entre o quanto um país aumentou suas emissões e o quanto ele aqueceu. O gráfico deixa isso evidente: **quase todos** os países apresentam tendência de temperatura **positiva** (+0,01 a +0,09 °C/ano), espalhados por toda a faixa de tendências de emissão. Os casos extremos são ilustrativos: **Rússia** e **Ucrânia** aqueceram entre os mais rápidos do mundo, mas suas emissões **caíram** (~ -17 e -13 Mt/ano) após o colapso da União Soviética nos anos 1990. Isso é cientificamente coerente: o CO₂ é um gás **globalmente bem misturado**, de modo que o aquecimento de um país é determinado pelo forçamento **global** (e por efeitos regionais, como a amplificação ártica), e não pelas suas emissões **locais**. A resposta honesta à pergunta, portanto, é que **não se observa correlação estatística entre o aumento das emissões de um país e o aumento da sua temperatura** — e a ausência de correlação é, ela própria, o achado relevante. Nota: a regressão de Pearson é sensível a *outliers* como a China (+205 Mt/ano), exibida fora de escala no gráfico.

5.7 P7 — Aceleração térmica (*Window Functions*)

O que foi feito: calculou-se a temperatura média por país e década e, com uma janela particionada por país e ordenada por década, usou-se `lag` para obter a temperatura da década anterior. A aceleração é a diferença entre a década de 2010 e a anterior; ordenou-se de forma decrescente.

```
1 df_decada_pais = (
2     df_filtrado.withColumn("Decada", (col("Year") / 10).cast("int") * 10)
3     .groupBy("Country", "Decada")
4     .agg(avg("AverageTemperature").alias("TempDecada"))
```

```

5 )
6
7 window_accel = Window.partitionBy("Country").orderBy("Decada")
8 df_accel = (
9     df_decada_pais.withColumn("TempAnterior", lag("TempDecada",
10         1).over(window_accel))
11     .withColumn("Aceleracao", col("TempDecada") - col("TempAnterior"))
12     .filter(col("Decada") == 2010)
13     .orderBy(col("Aceleracao").desc())
14     .limit(10)
15 )

```

Listing 15: Aceleração térmica por país via lag em Window Function.

Resultado (aceleração em °C entre a década de 2010 e a anterior):

- Finlândia (1,076), Rússia (1,049), Cazaquistão (0,778), Canadá (0,767)
- Irã (0,761), Iraque (0,736), Geórgia (0,727), Armênia (0,727)
- Turquia (0,663), Tajiquistão (0,639)

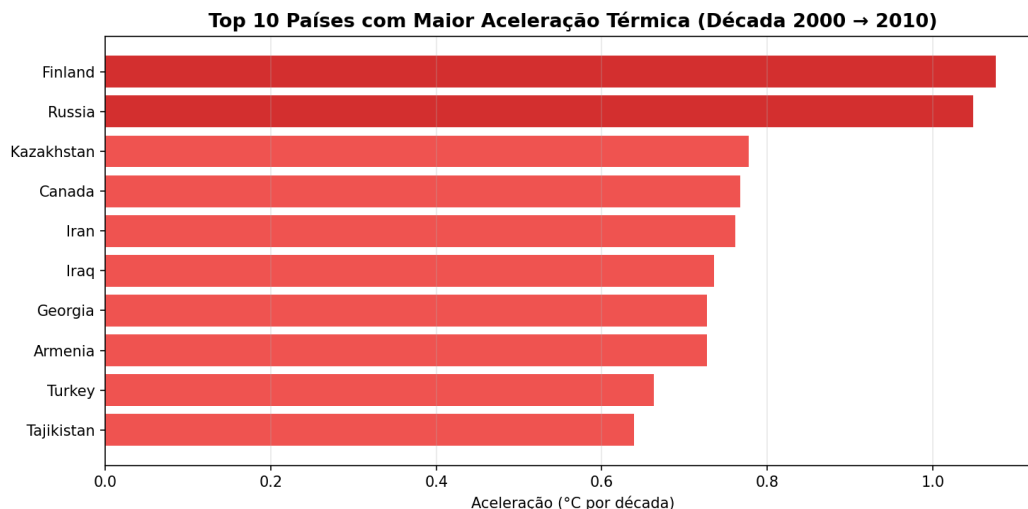


Figura 7: Top 10 países com maior aceleração térmica.

Interpretação: predominam países de **altas latitudes do hemisfério norte** (Finlândia, Rússia, Canadá) e do **Oriente Médio/Cáucaso**. O resultado é consistente com o fenômeno da **amplificação do Ártico**, em que regiões setentrionais aquecem mais rápido que a média global. Note que esta análise por *variação* (P7) é mais robusta que a correlação por níveis absolutos da P6.

5.8 P8 — Previsão de temperatura com MLlib

O que foi feito: para a cidade de São Paulo (1993–2013), montou-se um conjunto de treino com a temperatura média anual. Usando `VectorAssembler` (transformando o ano em vetor

de *features*) e `LinearRegression` do Spark MLlib, treinou-se um modelo e previram-se os anos de 2014 a 2018.

```
1 df_sp = (  
2     df_filtrado.filter((col("City") == "São Paulo") & (col("Year") >=  
3         1993))  
4     .groupBy("Year")  
5     .agg(avg("AverageTemperature").alias("label"))  
6     .orderBy("Year")  
7 )  
8 assembler = VectorAssembler(inputCols=["Year"], outputCol="features")  
9 df_ml = assembler.transform(df_sp).select("features", "label")  
10  
11 lr = LinearRegression(featuresCol="features", labelCol="label")  
12 lr_model = lr.fit(df_ml)  
13  
14 anos_futuros = spark.createDataFrame([(y,) for y in range(2014, 2019)],  
15     ["Year"])  
previsoes = lr_model.transform(assembler.transform(anos_futuros))
```

Listing 16: Regressão linear para previsão de temperatura em São Paulo.

Resultado: coeficiente de $-0,000099$ e intercepto de **20,79**, gerando previsão de $\sim 20,59^\circ\text{C}$ para 2014–2018.

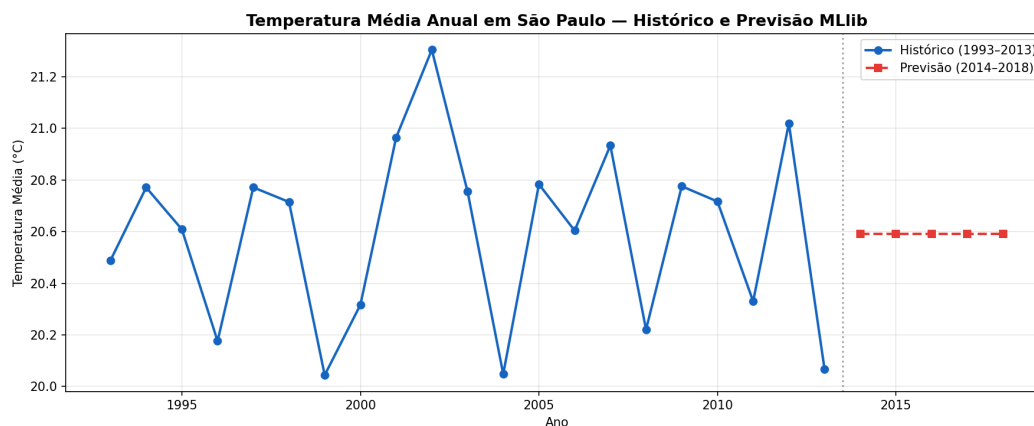


Figura 8: Temperatura histórica e previsão linear para São Paulo.

Interpretação: o coeficiente angular praticamente nulo indica uma tendência **estável** no período analisado para São Paulo. É importante reconhecer as **limitações**: o modelo é linear e simples, treinado com poucas amostras anuais (~ 21 pontos), o que o torna sensível a ruído e inadequado para extrapolações de longo prazo. Serve como demonstração da integração do MLlib ao pipeline, não como projeção climática rigorosa.

6 Evidências do Spark UI

O monitoramento das execuções foi feito pela **Spark UI** (porta 8080 no master) e pelo **History Server** (porta 18080), que preserva o histórico de aplicações concluídas. Os números a seguir foram **extraídos do log de eventos da execução real** da aplicação `AnaliseClimaticaGlobal` — a mesma que gerou os gráficos deste relatório —, cujo *pipeline* completo levou **~1 min 43 s**. As capturas de tela correspondentes devem ser inseridas manualmente pelo autor; os pontos a destacar em cada uma estão descritos abaixo.

6.1 DAG das operações mais pesadas

As operações de maior custo, identificadas pela aba **Stages**, são:

- **Leitura e materialização da base (532 MB):** a ingestão do CSV e a primeira contagem disparam estágios com **6 tarefas** (uma por core do cluster), que concentram o maior peso de E/S do *pipeline* (~10 s de leitura e ~13 s na contagem que materializa o cache).
- **P5 — *Window Function*:** o cálculo da média histórica particionada por City/Country exige um *shuffle* (operador `Exchange`) para reorganizar os dados por chave. Mediu-se um *shuffle* de **~105 MB** entre os estágios de escrita e leitura (a etapa P5 completa levou ~9 s).
- **P6 — *Join*:** o *inner join* por país/ano provoca *shuffle* de ambos os lados, confirmado no log como um `SortMergeJoin` (*shuffle* de **~134 MB**, e não um *broadcast*); a etapa P6 completa levou ~9 s.

6.2 Estágios, tempo de execução e tarefas

A execução analisada gerou **quase duas centenas de estágios** (numerados até ~187), dos quais muitos aparecem como SKIPPED — efeito da reutilização de resultados (*stage reuse*) e do *cache*. Pontos a comentar nas capturas:

- **Número de tarefas por estágio:** os estágios de leitura e agregação executam com **6 tarefas**, refletindo os **6 cores** dos 3 workers. Embora o valor lógico de `spark.sql.shuffle.partitions` seja 200, o ***Adaptive Query Execution (AQE)*** — ativo por padrão no Spark 3.5 — *coalesce* as partições pós-*shuffle* para apenas **6 a 7 tarefas**, pois o volume de dados após as agregações é pequeno. É esse valor reduzido (e não 200) que se observa na UI.
- **Tempo de execução e paralelismo:** a distribuição das tarefas entre os **3 workers** (6 cores no total) é visível na aba **Executors**, demonstrando o paralelismo efetivo do cluster.
- ***Shuffles* repetidos:** observam-se vários estágios reescrevendo ~128 MB de *shuffle* (a agregação `df_temp_country` de P6) recomputados a cada ação. Isso ocorre porque o *cache* foi aplicado a `df_limpo`, e não aos DataFrames já agregados/filtrados — uma oportunidade de otimização adicional.

- **Efeito do cache:** na aba **Storage**, o DataFrame `df_limpo` aparece como persistido em memória, confirmando a otimização discutida na Seção 4.

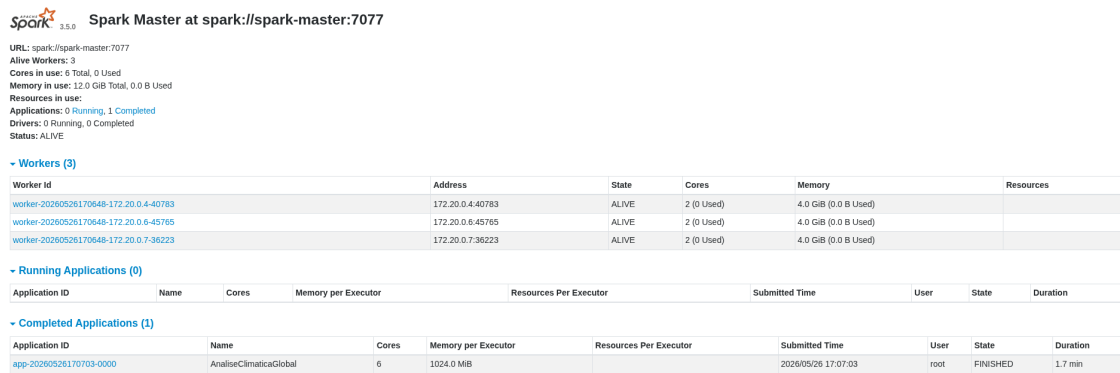


Figura 9: Spark Master UI (localhost:8080) com os 3 workers ativos.

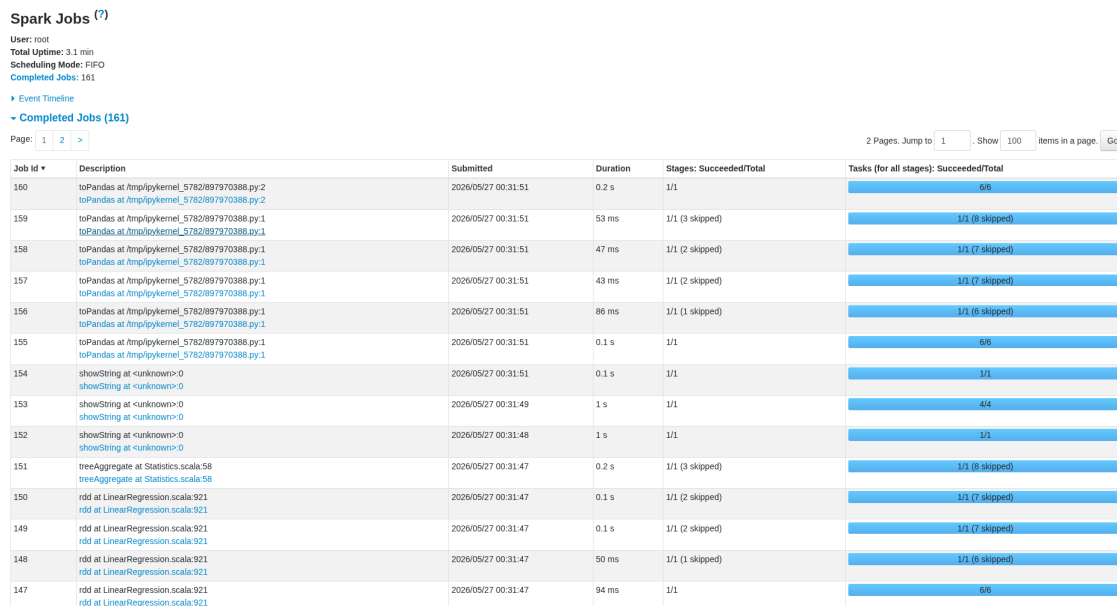
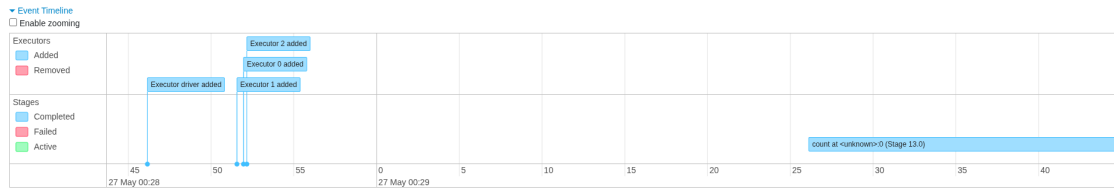


Figura 10: Aba Jobs da Spark UI depois da execução do pipeline.

Details for Job 10

Status: SUCCEEDED
Submitted: 2026/05/27 00:29:26
Duration: 19 s
Associated SQL Query: 0
Completed Stages: 1



DAG Visualization

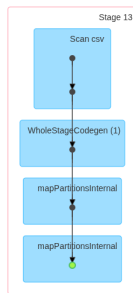


Figura 11: DAG e Event Timeline do stage mais pesado na Spark UI.

Stages for All Jobs

Completed Stages: 161
Skipped Stages: 170

Completed Stages (161)

Page: 1 2 >

2 Pages. Jump to 1. Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
330	toPandas at http://pykernel_5782/897970388.py:2	2026/05/27 00:31:51	0.1 s	6/6				
329	toPandas at http://pykernel_5782/897970388.py:1	2026/05/27 00:31:51	44 ms	1/1			1344.0 B	
325	toPandas at http://pykernel_5782/897970388.py:1	2026/05/27 00:31:51	37 ms	1/1			1533.0 B	1344.0 B
322	toPandas at http://pykernel_5782/897970388.py:1	2026/05/27 00:31:51	34 ms	1/1			1533.0 B	
319	toPandas at http://pykernel_5782/897970388.py:1	2026/05/27 00:31:51	78 ms	1/1			31.8 KiB	1533.0 B
317	toPandas at http://pykernel_5782/897970388.py:1	2026/05/27 00:31:51	0.1 s	6/6	273.6 MiB			31.8 KiB
316	showString at <unknown>:0	2026/05/27 00:31:51	0.1 s	1/1				
315	showString at <unknown>:0	2026/05/27 00:31:49	1 s	4/4				
314	showString at <unknown>:0	2026/05/27 00:31:48	1 s	1/1				
313	treeAggregate at Statistics.scala:58	2026/05/27 00:31:47	0.1 s	1/1			1344.0 B	
309	rdd at LinearRegression.scala:921	2026/05/27 00:31:47	73 ms	1/1			1533.0 B	1344.0 B
306	rdd at LinearRegression.scala:921	2026/05/27 00:31:47	90 ms	1/1			1533.0 B	
303	rdd at LinearRegression.scala:921	2026/05/27 00:31:47	40 ms	1/1			31.8 KiB	1533.0 B
301	rdd at LinearRegression.scala:921	2026/05/27 00:31:47	85 ms	6/6	273.6 MiB			31.8 KiB
300	treeAggregate at WeightedLeastSquares.scala:107	2026/05/27 00:31:46	0.7 s	1/1			1344.0 B	
296	rdd at LinearRegression.scala:348	2026/05/27 00:31:45	42 ms	1/1			1533.0 B	1344.0 B
293	rdd at LinearRegression.scala:348	2026/05/27 00:31:45	36 ms	1/1			1533.0 B	
290	rdd at LinearRegression.scala:348	2026/05/27 00:31:45	67 ms	1/1			31.8 KiB	1533.0 B
288	rdd at LinearRegression.scala:348	2026/05/27 00:31:45	48 ms	6/6	273.6 MiB			31.8 KiB
287	rdd at Instrumentation.scala:62	2026/05/27 00:31:45	33 ms	1/1			1533.0 B	1344.0 B
284	rdd at Instrumentation.scala:62	2026/05/27 00:31:45	51 ms	1/1			1533.0 B	
281	rdd at Instrumentation.scala:62	2026/05/27 00:31:44	96 ms	1/1			31.8 KiB	1533.0 B
279	rdd at Instrumentation.scala:62	2026/05/27 00:31:44	83 ms	6/6	273.6 MiB			31.8 KiB
278	showString at <unknown>:0	2026/05/27 00:31:44	55 ms	1/1			1533.0 B	
275	showString at <unknown>:0	2026/05/27 00:31:44	97 ms	1/1			31.8 KiB	1533.0 B
273	showString at <unknown>:0	2026/05/27 00:31:43	0.2 s	6/6	273.6 MiB			31.8 KiB
272	toPandas at http://pykernel_5782/1703294159.py:1	2026/05/27 00:31:43	41 ms	1/1			53.0 KiB	
268	toPandas at http://pykernel_5782/1703294159.py:1	2026/05/27 00:31:43	0.1 s	1/1			458.7 KiB	53.0 KiB

Figura 12: Aba Stages: distribuição de tarefas entre os executores.

RDD Storage Info for *(1) Project [dt#17, AverageTemperature#18, AverageTemperatureUncertainty#19, City#20, Country#21, cast(regex_replace(Latitude#22, [...

Storage Level: Disk Memory Deserialized 1x Replicated
 Cached Partitions: 6
 Total Partitions: 6
 Memory Size: 273.6 MiB
 Disk Size: 0.0 B

Data Distribution on 3 Executors

Host	On Heap Memory Usage	Off Heap Memory Usage	Disk Usage
172.18.0.6:43929	90.8 MiB (343.5 MiB Remaining)	0.0 B (0.0 B Remaining)	0.0 B
172.18.0.5:45263	90.8 MiB (343.5 MiB Remaining)	0.0 B (0.0 B Remaining)	0.0 B
172.18.0.4:36135	91.9 MiB (342.4 MiB Remaining)	0.0 B (0.0 B Remaining)	0.0 B

6 Partitions

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Block Name	Storage Level	Size in Memory	Size on Disk	Executors
rdd_43_5	Memory Deserialized 1x Replicated	44.3 MiB	0.0 B	172.18.0.6:43929
rdd_43_4	Memory Deserialized 1x Replicated	45.1 MiB	0.0 B	172.18.0.5:45263
rdd_43_3	Memory Deserialized 1x Replicated	45.8 MiB	0.0 B	172.18.0.4:36135
rdd_43_2	Memory Deserialized 1x Replicated	46.5 MiB	0.0 B	172.18.0.6:43929
rdd_43_1	Memory Deserialized 1x Replicated	45.7 MiB	0.0 B	172.18.0.5:45263
rdd_43_0	Memory Deserialized 1x Replicated	46.1 MiB	0.0 B	172.18.0.4:36135

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Figura 13: Aba Storage: distribuição de memória.

Executors
[Show Additional Metrics](#)

Summary

	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Excluded
Active(4)	0	0.0 B / 1.7 GiB	0.0 B	6	0	0	589	589	15 min (24 s)	13 GiB	2.4 GiB	2.4 GiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(4)	0	0.0 B / 1.7 GiB	0.0 B	6	0	0	589	589	15 min (24 s)	13 GiB	2.4 GiB	2.4 GiB	0

Executors
 Show 20 entries Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Add Time	Remove Time
driver	97f667acfe22:45275	Active	0	0.0 B / 434.4 MiB	0.0 B	0	0	0	0	0	3.1 min (0.0 ms)	0.0 B	0.0 B	0.0 B		2026-05-26 21:28:46	-
0	172.20.0.4:35777	Active	0	0.0 B / 434.4 MiB	0.0 B	2	0	0	180	180	4.0 min (9 s)	4.4 GiB	801.9 MiB	814.5 MiB	stdout stderr	2026-05-26 21:28:51	-
1	172.20.0.7:39623	Active	0	0.0 B / 434.4 MiB	0.0 B	2	0	0	196	196	4.0 min (8 s)	4.3 GiB	815.6 MiB	805.1 MiB	stdout stderr	2026-05-26 21:28:51	-
2	172.20.0.6:34043	Active	0	0.0 B / 434.4 MiB	0.0 B	2	0	0	213	213	4.1 min (8 s)	4.3 GiB	809.2 MiB	807.2 MiB	stdout stderr	2026-05-26 21:28:52	-

Figura 14: Aba Executors: 3 workers com tasks distribuídas.

7 Dicionário de Dados Final

O **dataset final enriquecido** produzido pelo *pipeline* é o `df_join` (construído na P6): o resultado do *inner join* entre a temperatura agregada por país/ano e a base de emissões de CO₂ já limpa, pelas chaves país e ano. Esse conjunto é persistido em `spark-data/output/dataset_final.csv`. Cada linha representa **um país em um ano**, combinando temperatura e emissões padronizadas. A Tabela 2 descreve cada coluna.

Coluna	Tipo	Descrição
Country	string	País (base de temperatura) — chave do <i>join</i> .
Year	int	Ano da medição — chave do <i>join</i> (<i>Year</i> >= 1974).
AvgTemp	double	Temperatura média anual do país (°C), agregada de <i>AverageTemperature</i> após limpeza.
Country_CO2	string	País na base de CO ₂ (OWID), sem agregados regionais — chave correspondente.
Year_CO2	int	Ano correspondente na base de CO ₂ .
co2	double	Emissões anuais de CO ₂ (milhões de toneladas, Mt).
co2_per_capita	double	Emissões de CO ₂ por habitante (t per capita).
total_ghg	double	Total de gases de efeito estufa (Mt CO ₂ eq).

Tabela 2: Dicionário de dados do *dataset* final (*df_join*).

As colunas intermediárias do ETL — *Month*, *Latitude/Longitude* (numéricas após a remoção de [NSEW]), *HistAvg* (média histórica por cidade usada no filtro de incerteza) e *Decada* — são utilizadas nas etapas anteriores, mas **não** compõem o *dataset* final do *join*.

Referências

- [Apa24] Apache Software Foundation. Apache spark — documentação oficial. <https://spark.apache.org>, 2024. Acesso em: maio 2026.
- [Ber17] Berkeley Earth. Climate change: Earth surface temperature data. <https://www.kaggle.com/datasets/berkeleyearth/climate-change-earth-surface-temperature-data>, 2017. Acesso em: maio 2026.
- [Cly23] Clynt. Apache spark with docker. <https://clynt.com/blog/data-engineering/apache-spark/apache-spark-with-docker>, 2023. Acesso em: maio 2026.
- [Kul20] Pavan P. Kulkarni. Spark on docker — cluster com history server. <https://www.pavanpkulkarni.com/blog/13-spark-on-docker/>, 2020. Acesso em: maio 2026.
- [Mat24a] Material da disciplina. Apache spark — vídeo complementar (indicado pelo professor). <https://www.youtube.com/watch?v=IUSuyx2fMCU>, 2024. Acesso em: maio 2026.
- [Mat24b] Material da disciplina. Apache spark — vídeo introdutório (indicado pelo professor). <https://www.youtube.com/watch?v=Iema3-fSo7g>, 2024. Acesso em: maio 2026.
- [Our24] Our World in Data. CO2 and greenhouse gas emissions. <https://github.com/owid/co2-data>, 2024. Acesso em: maio 2026.

- [Sap22] Eric Saphé. Testing apache spark locally: Docker compose and kubernetes deployment. <https://medium.com/@SaphE/testing-apache-spark-locally-docker-compose-and-kubernetes-deployment-94d35a54f> 2022. Acesso em: maio 2026.
- [Wit21] Wittline. Apache spark docker — cluster com jupyter e hdfs. <https://wittline.github.io/apache-spark-docker/>, 2021. Acesso em: maio 2026.